

# SOB4ES - Procesado de datos de consultas remotas a bases de datos (APIs)

## CRISP-ML(Q) Fase 2: Ingeniería de Datos

Este notebook extrae variables ambientales adicionales para los 428 sitios SOB4ES usando múltiples fuentes de datos online.

Sigue el mismo procedimiento que `01_datos_locales.ipynb`: registro → extracción vectorizada → cobertura → límites → exportación.

| # | Fuente               | API / Colección                    | Variables                                                | Formato salida |
|---|----------------------|------------------------------------|----------------------------------------------------------|----------------|
| 1 | Google Earth Engine  | ECMWF/ERA5_LAND/DAILY_AGGR         | Temperatura media anual (°C), Humedad relativa media (%) | Numérico       |
| 2 | Google Earth Engine  | COPERNICUS/S2_SR_HARMONIZED        | NDVI (media verano)                                      | Numérico       |
| 3 | Copernicus CDS       | reanalysis-era5-land-monthly-means | Precipitación mensual media (mm/mes)                     | Numérico       |
| 4 | Copernicus DEM (AWS) | COG tiles vía /vsicurl/            | Elevación (m), Pendiente (°), Orientación (°)            | Numérico       |

**Salida:** `output/online_features.csv` — 428 filas × N columnas, lista para unir con la salida del notebook 01.

## 0.- Configuración del Entorno

```
In [43]: # Importaciones y configuración de rutas

# Dependencias – instalar si no están disponibles:
# pip install earthengine-api cdsapi xarray rasterio tqdm pandas numpy

import os, sys, math, warnings
import numpy as np
import pandas as pd
import xarray as xr
import rasterio
from rasterio.crs import CRS
from rasterio.warp import transform as rio_transform
from tqdm.notebook import tqdm
import ee
import cdsapi

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 40)
pd.set_option('display.float_format', '{:.4f}'.format)

WGS84 = CRS.from_epsg(4326)

SITES_CSV = 'output/sob4es_clean_v14.csv' # Actualizar en base a la ultima versión del export local

CDS_DIR = 'output/cds_downloads/'
OUT_DIR = 'output/'

os.makedirs(CDS_DIR, exist_ok=True)
os.makedirs(OUT_DIR, exist_ok=True)

# Verificación
for nombre, ruta in [('SITES_CSV', SITES_CSV), ('CDS_DIR', CDS_DIR), ('OUT_DIR', OUT_DIR)]:
    existe = os.path.exists(ruta)
    estado = 'Archivos encontrados' if existe else '[ERROR] Archivo no encontrado'
    print(f' {nombre}: {ruta} {estado}')
```

SITES\_CSV: output/sob4es\_clean\_v14.csv Archivos encontrados  
CDS\_DIR: output/cds\_downloads/ Archivos encontrados  
OUT\_DIR: output/ Archivos encontrados

```
In [44]: # Autenticación APIs

# 1. Google Earth Engine
# Una sola vez por máquina (no por sesión):
# a) Regístrate en https://earthengine.google.com y solicita acceso.
# b) En terminal: earthengine authenticate
# → abre el navegador, pide permiso, guarda credenciales en:
# ~/.config/earthengine/credentials (local)

try:
    ee.Initialize()
    print('Iniciando Google Earth Engine...')
except Exception as e:
    print(f'[WARN] GEE no autenticado: {e}')
    print('\t Ejecuta ee.Authenticate() y vuelve a intentarlo.')

# 2. Copernicus Climate Data Store
# Una sola vez por máquina:
# a) Crea cuenta en https://cds.climate.copernicus.eu
# b) Perfil → API key → copia la clave.
# c) Crea ~/.cdsapirc con:
# url: https://cds.climate.copernicus.eu/api
# key: <tu-api-key>

try:
    cds_client = cdsapi.Client(quiet=True)
    print('Iniciando Copernicus CDS...')
except Exception as e:
    print(f'[WARN] CDS no configurado: {e}')
    print('\t Crea ~/.cdsapirc con tu API key (ver instrucciones arriba).')
```

# 3. Copernicus DEM (AWS)  
# Sin autenticación mediante tiles públicos vía HTTPS.  
# rasterio accede con /vsicurl/ (requiere GDAL con soporte curl, incluido por defecto).  
print('Copernicus DEM (AWS), sin autenticación requerida')

Iniciando Google Earth Engine...  
Iniciando Copernicus CDS...  
Copernicus DEM (AWS), sin autenticación requerida

## 1.- Cargar Coordenadas de Sitios

Las coordenadas se leen desde la salida del notebook 01 ( `sob4es_clean_vx.csv` ).

Solo necesitamos `site_id`, `latitude` y `longitude`, el resto del pipeline es independiente.

```
In [45]: sites = pd.read_csv(SITES_CSV, usecols=['site_id', 'latitude', 'longitude'])

lats = sites['latitude'].to_numpy()
lons = sites['longitude'].to_numpy()

print(f'Sitios cargados: {len(sites)}')
print(f'Rango lat: {lats.min():.2f} - {lats.max():.2f}')
print(f'Rango lon: {lons.min():.2f} - {lons.max():.2f}')
sites.head(3)
```

```
Sitios cargados: 428
Rango lat: 31.36 - 60.13
Rango lon: -8.68 - 35.27
```

```
Out[45]:
```

|   | site_id | latitude | longitude |
|---|---------|----------|-----------|
| 0 | BE_001  | 51.0990  | 4.9750    |
| 1 | BE_002  | 51.1020  | 4.9780    |
| 2 | BE_003  | 51.0970  | 4.9760    |

## 2.- Google Earth Engine

### 2.1.- Registro de Colecciones GEE

Mismo patrón que `EU_RASTERS` en el notebook 01: un diccionario central con metadatos de cada variable.

Las funciones de extracción se definen en la sección 2.2.

```
In [46]: # Período de referencia
# Ajusta según el período de muestreo del proyecto SOB4ES.
ERAS_START = '2015-01-01'
ERAS_END = '2020-12-31'
NDVI_START = '2019-06-01' # verano para maximizar cobertura vegetal
NDVI_END = '2019-09-01'
NDVI_CLOUD = 20 # máximo % de nubosidad aceptado en Sentinel-2

# Registro GEE
# Formato: 'nombre_col': (colección_GEE, banda(s), descripción, unidad, (lo, hi))
# Las funciones de extracción usan este dict para saber qué pedir a la API.
GEE_LAYERS = {
    # ERA5-Land: temperatura media anual (K → °C en extracción)
    'gee_temp_media_C' : ('ECMWF/ERA5_LAND/DAILY_AGGR',
                          ['temperature_2m'],
                          f'Temperatura media anual {ERAS_START[:4]}-{ERAS_END[:4]}',
                          '°C', (-60.0, 60.0)),
    # ERA5-Land: humedad relativa media anual (calculada de temp + dew point)
    'gee_humedad_rel_pct': ('ECMWF/ERA5_LAND/DAILY_AGGR',
                            ['temperature_2m', 'dewpoint_temperature_2m'],
                            f'Humedad relativa media anual {ERAS_START[:4]}-{ERAS_END[:4]}',
                            '%', (0.0, 100.0)),
    # Sentinel-2: NDVI medio de verano
    'gee_ndvi_verano' : ('COPERNICUS/S2_SR_HARMONIZED',
                        ['B8', 'B4'],
                        f'NDVI medio verano {NDVI_START[:7]} - {NDVI_END[:7]}',
                        'índice', (-1.0, 1.0)),
    # Añadir nuevas variables GEE a partir de aquí
    # 'gee_XXX': ('COLECCION/GEE', ['banda'], 'descripción', 'unidad', (lo, hi)),
}

print(f'{len(GEE_LAYERS)} variables GEE registradas:')
for col, (col_id, bands, desc, unit, bounds) in GEE_LAYERS.items():
    print(f' {col:30s} {unit:7s} límites={bounds}')
    print(f' Colección: {col_id} | Bandas: {bands}')

3 variables GEE registradas:
gee_temp_media_C          °C          límites=(-60.0, 60.0)
Colección: ECMWF/ERA5_LAND/DAILY_AGGR | Bandas: ['temperature_2m']
gee_humedad_rel_pct       %           límites=(0.0, 100.0)
Colección: ECMWF/ERA5_LAND/DAILY_AGGR | Bandas: ['temperature_2m', 'dewpoint_temperature_2m']
gee_ndvi_verano           índice       límites=(-1.0, 1.0)
Colección: COPERNICUS/S2_SR_HARMONIZED | Bandas: ['B8', 'B4']
```

### 2.2.- Funciones de Extracción GEE

Estrategia de extracción: `ee.FeatureCollection + reduceRegions()` — extrae todos los sitios en **una sola llamada** a la API en lugar de 428 llamadas individuales.

```
In [47]: # Helpers GEE

def _sites_to_fc(lats, lons, site_ids):
    """Convierte arrays de coordenadas a ee.FeatureCollection para reduceRegions."""
    features = [
        ee.Feature(ee.Geometry.Point([float(lon), float(lat)]),
                    {'SITE_ID': sid})
        for lat, lon, sid in zip(lats, lons, site_ids)
    ]
    return ee.FeatureCollection(features)

def _fc_to_df(fc, value_col, id_col='SITE_ID'):
    """
    Descarga los resultados de un reduceRegions como DataFrame.
    Convierte a float; los sitios sin dato quedan como NaN.
    """
    data = fc.getInfo()['features']
    return pd.DataFrame([
        {id_col: f['properties'].get(id_col),
         value_col: f['properties'].get('mean')}
        for f in data
    ])

def _add_rh_band(img):
    """
    Añade banda de humedad relativa usando la fórmula de Magnus:
    RH = 100 × exp(17.625×Td/(243.04+Td)) / exp(17.625×T/(243.04+T))
    donde T y Td están en °C.
    """
    T = img.select('temperature_2m').subtract(273.15)
    Td = img.select('dewpoint_temperature_2m').subtract(273.15)
    e = lambda x: ee.Image(math.e).pow(x.multiply(17.625).divide(x.add(243.04)))
    RH = ee.Image(100).multiply(e(Td).divide(e(T))).rename('relative_humidity')
```

```
return img.addBands(RH)

print('Helpers GEE definidos')
```

Helpers GEE definidos

## 2.3.- Extracción ERA5-Land (Temperatura y Humedad Relativa)

```
In [48]: # ERA5-Land: temperatura media y humedad relativa
# Una sola llamada reduceRegions para todos los sitios.
# Escala: 11132 m (resolución nativa ERA5-Land ~0.1°)

fc_sites = _sites_to_fc(lats, lons, sites['site_id'])

era5_col = (
    ee.ImageCollection('ECMWF/ERA5_LAND/DAILY_AGGR')
    .filterDate(ERA5_START, ERA5_END)
    .select(['temperature_2m', 'dewpoint_temperature_2m'])
    .map(_add_rh_band)
    .mean() # media temporal sobre todo el periodo
)

# Temperatura: convertir K → °C
temp_img = era5_col.select('temperature_2m').subtract(273.15)
rh_img = era5_col.select('relative_humidity')

# reduceRegions: extrae valor en el pixel que contiene cada punto
temp_fc = temp_img.reduceRegions(collection=fc_sites, reducer=ee.Reducer.mean(), scale=11132)
rh_fc = rh_img.reduceRegions(collection=fc_sites, reducer=ee.Reducer.mean(), scale=11132)

df_temp = _fc_to_df(temp_fc, 'gee_temp_media_C')
df_rh = _fc_to_df(rh_fc, 'gee_humedad_rel_pct')

df_era5 = df_temp.merge(df_rh, on='SITE_ID', how='outer')

print(f'ERA5 extraído: {df_era5.shape}')
print(f' Temperatura media: {df_era5["gee_temp_media_C"].mean():.2f}°C')
print(f' Hum. relativa media: {df_era5["gee_humedad_rel_pct"].mean():.1f}%')
df_era5.head(3)
```

ERA5 extraído: (428, 3)  
Temperatura media: 11.91°C  
Hum. relativa media: 75.0%

```
Dut[48]:
```

|   | SITE_ID | gee_temp_media_C | gee_humedad_rel_pct |
|---|---------|------------------|---------------------|
| 0 | BE_001  | 11.6396          | 75.7536             |
| 1 | BE_002  | 11.5443          | 76.1178             |
| 2 | BE_003  | 11.6396          | 75.7536             |

## 2.4.- Extracción Sentinel-2 NDVI

```
In [49]: # Sentinel-2: NDVI medio de verano
# Filtro de nubes: CLOUDY_PIXEL_PERCENTAGE < NDVI_CLOUD
# Escala: 10 m (resolución nativa Sentinel-2 bandas B4/B8)

s2_col = (
    ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterDate(NDVI_START, NDVI_END)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', NDVI_CLOUD))
    .map(lambda img: img.normalizedDifference(['B8', 'B4']))
    .rename('NDVI')
    .copyProperties(img, ['system:time_start'])
    .mean()
)

ndvi_fc = s2_col.reduceRegions(collection=fc_sites, reducer=ee.Reducer.mean(), scale=10)
df_ndvi = _fc_to_df(ndvi_fc, 'gee_ndvi_verano')

print(f'NDVI extraído: {df_ndvi.shape}')
print(f' NDVI medio: {df_ndvi["gee_ndvi_verano"].mean():.4f}')
print(f' NaN (sin imagen): {df_ndvi["gee_ndvi_verano"].isna().sum()}')
df_ndvi.head(3)
```

NDVI extraído: (428, 2)  
NDVI medio: 0.6032  
NaN (sin imagen): 9

```
Dut[49]:
```

|   | SITE_ID | gee_ndvi_verano |
|---|---------|-----------------|
| 0 | BE_001  | 0.7096          |
| 1 | BE_002  | 0.5699          |
| 2 | BE_003  | 0.6483          |

## 2.5.- Cobertura y Límites GEE

```
In [50]: # Unir variables GEE
df_gee = df_era5.merge(df_ndvi, on='SITE_ID', how='outer')

# Cobertura
gee_cols = [c for c in df_gee.columns if c != 'SITE_ID']
cobertura_gee = (
    df_gee[gee_cols].notna().sum() / len(df_gee) * 100
).round(1).rename('cobertura_%').to_frame()
cobertura_gee['n_validos'] = df_gee[gee_cols].notna().sum()
cobertura_gee['n_nulos'] = df_gee[gee_cols].isna().sum()
print('Cobertura GEE:')
print(cobertura_gee.to_string())

# Límites de dominio
flag_gee = []
for col, (_, _, unit, (lo, hi)) in GEE_LAYERS.items():
    if col not in df_gee.columns:
        continue
    mask = pd.Series(False, index=df_gee.index)
    if lo is not None: mask |= df_gee[col] < lo
    if hi is not None: mask |= df_gee[col] > hi
    n = mask.sum()
    if n:
        flag_gee.append({'columna': col, 'unidad': unit,
```

```

        'n_fuera_rango': n, 'rango_valido': f'[{lo}, {hi}]'})
df_gee[col] = df_gee[col].clip(
    lower=lo if lo is not None else -np.inf,
    upper=hi if hi is not None else np.inf
)

if flag_gee:
    print('\nValores fuera de rango detectados y recortados:')
    print(pd.DataFrame(flag_gee).to_string(index=False))
else:
    print('\nTodos los límites GEE superados...')

```

Cobertura GEE:

|                     | cobertura_% | n_validos | n_nulos |
|---------------------|-------------|-----------|---------|
| gee_temp_media_C    | 99.8000     | 427       | 1       |
| gee_humedad_rel_pct | 99.8000     | 427       | 1       |
| gee_ndvi_verano     | 97.9000     | 419       | 9       |

Todos los límites GEE superados...

### 3.- Copernicus Climate Data Store (Precipitación)

Dataset: reanalysis-era5-land-monthly-means

Variable: total\_precipitation (m/mes en ERA5 → convertida a mm/mes)

Estrategia: descargar archivos .nc por año → abrir como dataset multiannual con xarray → extraer por coordenada.

#### 3.1.- Registro CDS

```

In [51]: # Configuración CDS
CDS_AÑOS = list(range(2015, 2021)) # ajusta al periodo de muestreo S0B4ES
CDS_DATASET = 'reanalysis-era5-land-monthly-means'

# Registro CDS
# Formato: 'nombre_col': (variable_era5, factor_conversion, unidad, (lo, hi))
CDS_LAYERS = {
    'cds_precip_mm_mes': ('total_precipitation', 1000.0, 'mm/mes', (0.0, 2000.0)),
    # Añadir más variables CDS a partir de aquí
    # ERA5-Land mensual ofrece también: temperatura_2m, evapotranspiración, etc.
    # 'cds_XXX': ('nombre_variable_era5', factor, 'unidad', (lo, hi)),
}

print(f'{len(CDS_LAYERS)} variables CDS registradas:')
for col, (var, factor, unit, bounds) in CDS_LAYERS.items():
    print(f' {col:30s} variable ERA5={var} ×{factor} {unit} límites={bounds}')

```

1 variables CDS registradas:

cds\_precip\_mm\_mes                      variable ERA5=total\_precipitation    ×1000.0 mm/mes    límites=(0.0, 2000.0)

#### 3.2.- Descarga de Archivos NetCDF

```

In [52]: import zipfile
import netCDF4

# CDS_VAR_NAMES: mapeo nombre largo (API) → nombre corto en el .nc
# El API de CDS acepta 'total_precipitation' pero el .nc usa 'tp' (CF convention).
CDS_VAR_NAMES = {
    'total_precipitation' : 'tp',
    'temperature_2m' : 't2m',
    '2m_dewpoint_temperature' : 'd2m',
    'evaporation' : 'e',
}

def descargar_cds_año(año, variables, output_dir):
    """
    Descarga medias mensuales ERA5-Land para un año.
    'variables' son nombres largos del API CDS (ej. 'total_precipitation').
    Salta si el archivo ya existe y es un NetCDF4 válido.

    El API de CDS a veces devuelve un ZIP en lugar de un .nc suelto
    (cabecera PK en lugar de HDF). Se detecta y descomprime automáticamente.
    """
    fname = os.path.join(output_dir, f'era5_land_{año}.nc')

    # Si ya existe, verificar que es un NetCDF4 válido (cabecera HDF5 = b'\x89HDF')
    if os.path.exists(fname):
        with open(fname, 'rb') as fh:
            header = fh.read(4)
            if header == b'\x89HDF':
                print(f' {año}: ya existe, omitido.')
                return fname
        # Archivo corrupto o ZIP de descarga anterior – eliminar y repetir
        os.remove(fname)
        print(f' {año}: archivo inválido eliminado, re-descargando...')

    zip_path = fname.replace('.nc', '_tmp.zip')
    cds_client.retrieve(
        CDS_DATASET,
        {
            'product_type' : 'monthly_averaged_reanalysis',
            'variable' : variables,
            'year' : str(año),
            'month' : [f'{m:02d}' for m in range(1, 13)],
            'time' : '00:00',
            'format' : 'netcdf',
            'download_format' : 'unarchived', # pide archivo suelto, no ZIP
        },
        zip_path,
    )

    # Comprobar si el CDS igualmente devolvió un ZIP
    with open(zip_path, 'rb') as fh:
        is_zip = fh.read(4) == b'PK\x03\x04'

    if is_zip:
        with zipfile.ZipFile(zip_path) as zf:
            nc_inside = [n for n in zf.namelist() if n.endswith('.nc')][0]
            with zf.open(nc_inside) as src, open(fname, 'wb') as dst:
                dst.write(src.read())
        os.remove(zip_path)
    print(f' {año}: descomprimido → {fname}')

```

```

else:
    os.rename(zip_path, fname)
    print(f' {año}: descargado → {fname}')

return fname

# Eliminar archivos ZIP corruptos de descargas anteriores antes de empezar
for _f in [os.path.join(CDS_DIR, f'era5_land_{a}.nc') for a in CDS_AÑOS]:
    if os.path.exists(_f):
        with open(_f, 'rb') as _fh:
            if _fh.read(4) == b'PK\x03\x04':
                os.remove(_f)
                print(f' Eliminado archivo ZIP inválido: {_f}')

# Variables únicas en nombre largo (sin duplicados)
vars_cds_api = list({v for v, _, _, _ in CDS_LAYERS.values()})

archivos_nc = [
    descargar_cds_año(año, vars_cds_api, CDS_DIR)
    for año in tqdm(CDS_AÑOS, desc='Descarga CDS')
]

# Abrir como dataset multianual con engine explícito
import xarray as xr

datasets = [xr.open_dataset(f, engine='netcdf4') for f in archivos_nc]
ds_cds = xr.concat(datasets, dim='valid_time') if 'valid_time' in datasets[0].dims else xr.concat(datasets, dim='time')
print(f'\nDataset CDS abierto: {dict(ds_cds.dims)}')
print(f'Variables en .nc: {list(ds_cds.data_vars)}')

# Verificar mapeo CDS_VAR_NAMES
print('\nMapeo API → .nc:')
for largo, corto in CDS_VAR_NAMES.items():
    if largo in vars_cds_api:
        ok = corto in ds_cds.data_vars
        print(f' {largo:30s} → {corto:6s} {"CDS mapeados" if ok else "CDS no encontrados, revisa CDS_VAR_NAMES"}')

```

Descarga CDS: 0%| | 0/6 [00:00<?, ?it/s]

2015: ya existe, omitido.  
2016: ya existe, omitido.  
2017: ya existe, omitido.  
2018: ya existe, omitido.  
2019: ya existe, omitido.  
2020: ya existe, omitido.

Dataset CDS abierto: {'valid\_time': 72, 'latitude': 1801, 'longitude': 3600}  
Variables en .nc: ['tp']

Mapeo API → .nc:  
total\_precipitation → tp CDS mapeados

### 3.3.- Extracción por Coordenada

```

In [53]: # Extracción vectorizada CDS

cds_records = {'site_id': sites['site_id'].tolist()}

for col, (var_name, factor, unit, bounds) in tqdm(
    CDS_LAYERS.items(), desc='Extracción CDS', total=len(CDS_LAYERS)):

    valores = []
    for lat, lon in zip(lats, lons):
        try:
            val = float(
                ds_cds[var_name]
                .sel(latitude=lat, longitude=lon % 360, method='nearest')
                .mean() # media temporal sobre todos los meses descargados
                .values
            ) * factor # conversión de unidades (e.g. m/día → mm/día)
        except Exception:
            val = float('nan')
            valores.append(val)
        cds_records[col] = valores

df_cds = pd.DataFrame(cds_records)
print(f'CDS extraído: {df_cds.shape}')
for col in [c for c in df_cds.columns if c != 'site_id']:
    media = df_cds[col].mean()
    nulos = df_cds[col].isna().sum()
    print(f' {col}: media={media:.2f} NaN={nulos}')
df_cds.head(3)

```

Extracción CDS: 0%| | 0/1 [00:00<?, ?it/s]

CDS extraído: (428, 2)  
cds\_precip\_mm\_mes: media=nan NaN=428

Out[53]:

|   | site_id | cds_precip_mm_mes |
|---|---------|-------------------|
| 0 | BE_001  | NaN               |
| 1 | BE_002  | NaN               |
| 2 | BE_003  | NaN               |

```

In [54]: # Cobertura y límites CDS
cds_cols = [c for c in df_cds.columns if c != 'site_id']
cobertura_cds = (
    df_cds[cds_cols].notna().sum() / len(df_cds) * 100
).round(1).rename('cobertura_%').to_frame()
cobertura_cds['n_validos'] = df_cds[cds_cols].notna().sum()
cobertura_cds['n_nulos'] = df_cds[cds_cols].isna().sum()
print('Cobertura CDS:')
print(cobertura_cds.to_string())

flag_cds = []
for col, (_, _, unit, (lo, hi)) in CDS_LAYERS.items():
    if col not in df_cds.columns:
        continue
    mask = pd.Series(False, index=df_cds.index)
    if lo is not None: mask |= df_cds[col] < lo
    if hi is not None: mask |= df_cds[col] > hi
    n = mask.sum()
    if n:
        flag_cds.append({'columna': col, 'unidad': unit,

```

```

        'n_fuera_rango': n, 'rango_valido': f'[{lo}, {hi}]'})
df_cds[col] = df_cds[col].clip(
    lower=lo if lo is not None else -np.inf,
    upper=hi if hi is not None else np.inf
)

if flag_cds:
    print('\nValores fuera de rango detectados y recortados:')
    print(pd.DataFrame(flag_cds).to_string(index=False))
else:
    print('\nTodos los límites CDS superados...')

```

Cobertura CDS:

|                   | cobertura_% | n_validos | n_nulos |
|-------------------|-------------|-----------|---------|
| cds_precip_mm_mes | 0.0000      | 0         | 428     |

Todos los límites CDS superados...

## 4.- Copernicus DEM — Elevación, Pendiente y Orientación

Los tiles del DEM de 30 m están disponibles públicamente en AWS S3. rasterio los lee directamente vía `/vsicurl/` sin descargar el archivo completo.

Pendiente y orientación se derivan del DEM usando diferencias centrales (método Horn 1981) sobre una ventana 3×3 píxeles alrededor de cada punto.

### 4.1.- Registro DEM

```

In [55]: # Registro DEM
# Formato: 'nombre_col': (descripción, unidad, (lo, hi))
# Las tres columnas se extraen siempre juntas (una apertura de tile por sitio).
DEM_LAYERS = {
    'dem_elevacion_m' : ('Elevación sobre el nivel del mar', 'm', (-500.0, 9000.0)),
    'dem_pendiente_deg' : ('Pendiente del terreno', '°', (0.0, 90.0)),
    'dem_orientacion_deg': ('Orientación de la pendiente (N=0°)', '°', (0.0, 360.0)),
    # Añadir más derivadas del DEM a partir de aquí
    # (requeriría modificar la función dem_features_punto)
}

DEM_BUCKET = 'https://copernicus-dem-30m.s3.amazonaws.com'
DEM_WINDOW = 1 # radio en píxeles alrededor del punto central (ventana 3×3)

print(f'{len(DEM_LAYERS)} variables DEM registradas:')
for col, (desc, unit, bounds) in DEM_LAYERS.items():
    print(f' {col:30s} {unit:3s} límites={bounds}')
    print(f' {desc}')

```

3 variables DEM registradas:

|                                    |   |                          |
|------------------------------------|---|--------------------------|
| dem_elevacion_m                    | m | Límites=(-500.0, 9000.0) |
| Elevación sobre el nivel del mar   |   |                          |
| dem_pendiente_deg                  | ° | Límites=(0.0, 90.0)      |
| Pendiente del terreno              |   |                          |
| dem_orientacion_deg                | ° | Límites=(0.0, 360.0)     |
| Orientación de la pendiente (N=0°) |   |                          |

### 4.2.- Funciones de Extracción DEM

```

In [56]: # Helpers DEM

def _url_tile_dem(lat, lon):
    """
    Construye la URL S3 del tile Copernicus DEM 30m que contiene el punto (lat, lon).
    Formato tile: Copernicus_DSM_C06_10_N{lat:02d}_00_E{lon:03d}_00_DEM
    Documentación: https://registry.opendata.aws/copernicus-dem/
    """
    lf = int(math.floor(lat))
    lo = int(math.floor(lon))
    ns = 'N' if lf >= 0 else 'S'
    ew = 'E' if lo >= 0 else 'W'
    tile = (f'Copernicus_DSM_C06_10_{ns}{abs(lf):02d}_00'
            f'_{ew}{abs(lo):03d}_00_DEM')
    return f'{DEM_BUCKET}/{tile}/{tile}.tif'

def _pendiente_orientacion(ventana, res_m):
    """
    Calcula pendiente (°) y orientación (°) desde una ventana 3×3 del DEM.
    Método: diferencias centrales de Horn (1981).
    dz/dx = (E - W) / (2 * res_m)
    dz/dy = (N - S) / (2 * res_m) [N = fila 0, S = fila 2]
    """
    dz_dx = (ventana[1, 2] - ventana[1, 0]) / (2 * res_m)
    dz_dy = (ventana[0, 1] - ventana[2, 1]) / (2 * res_m)
    pendiente = math.degrees(math.atan(math.sqrt(dz_dx**2 + dz_dy**2)))
    orientacion = math.degrees(math.atan2(-dz_dx, dz_dy)) % 360
    return pendiente, orientacion

def dem_features_punto(lat, lon):
    """
    Extrae elevación (m), pendiente (°) y orientación (°) para un punto.
    Lee el tile C06 directamente desde AWS vía /vsicurl/ — sin descarga.
    Devuelve dict con NaN si el tile no existe o el punto está fuera de bounds.
    """
    NAN = {'dem_elevacion_m': np.nan,
          'dem_pendiente_deg': np.nan,
          'dem_orientacion_deg': np.nan}
    url = _url_tile_dem(lat, lon)
    try:
        with rasterio.open(f'/vsicurl/{url}') as src:
            # Reprojectar lat/lon WGS84 al CRS del tile (normalmente WGS84 también)
            xs, ys = rio_transform(WGS84, src.crs, [lon], [lat])
            from rasterio.transform import rowcol
            r, c = rowcol(src.transform, xs[0], ys[0])

            w = DEM_WINDOW
            r0, c0 = max(0, r-w), max(0, c-w)
            win = rasterio.windows.Window(c0, r0, 2*w+1, 2*w+1)
            data = src.read(1, window=win).astype(float)

            if data.shape != (2*w+1, 2*w+1):
                return NAN # borde del tile

            elev = float(data[w, w])
            if src.nodata and np.isclose(elev, src.nodata, rtol=1e-3):

```

```

        return NAN

    # Resolución en metros (aproximación para WGS84)
    res_deg = src.res[0]
    res_m = res_deg * 111320 * math.cos(math.radians(lat))
    pendiente, orientacion = _pendiente_orientacion(data, res_m)

    return {'dem_elevacion_m': round(elev, 2),
            'dem_pendiente_deg': round(pendiente, 4),
            'dem_orientacion_deg': round(orientacion, 4)}

except Exception:
    return NAN

print('Funciones DEM definidas')

```

Funciones DEM definidas

### 4.3.- Extracción por Lotes

```

In [57]: # Extracción DEM – un tile por sitio, leído vía /vsicurl/
# Nota: cada apertura de /vsicurl/ hace una petición HTTP al tile de AWS.
# Los tiles adyacentes se cachean por GDAL, así que sitios cercanos son rápidos.
# Para 428 sitios europeos (~100-200 tiles distintos): ~2-5 min.

dem_records = [
    dem_features.punto(lat, lon)
    for lat, lon in tqdm(zip(lats, lons), total=len(lats), desc='DEM (AWS C06)')
]

df_dem = pd.DataFrame(dem_records, index=sites.index)
df_dem.insert(0, 'site_id', sites['site_id'].values)

print(f'DEM extraído: {df_dem.shape}')
print(f' Elevación media : {df_dem["dem_elevacion_m"].mean():.0f} m')
print(f' Pendiente media : {df_dem["dem_pendiente_deg"].mean():.2f}°')
df_dem.head(3)

```

```

DEM (AWS C06): 0% | 0/428 [00:00<?, ?it/s]
DEM extraído: (428, 4)
Elevación media : 221 m
Pendiente media : 5.09°

```

```

Out[57]:
  site_id  dem_elevacion_m  dem_pendiente_deg  dem_orientacion_deg
0  BE_001             13.3100             0.4745             93.2040
1  BE_002             12.4300             0.1612             131.8142
2  BE_003             12.8300             3.4032             230.9894

```

```

In [58]: # Cobertura y límites DEM
dem_cols = [c for c in df_dem.columns if c != 'site_id']
cobertura_dem = (
    df_dem[dem_cols].notna().sum() / len(df_dem) * 100
).round(1).rename('cobertura_%').to_frame()
cobertura_dem['n_validos'] = df_dem[dem_cols].notna().sum()
cobertura_dem['n_nulos'] = df_dem[dem_cols].isna().sum()
print('Cobertura DEM:')
print(cobertura_dem.to_string())

flag_dem = []
for col, (_, unit, (lo, hi)) in DEMLAYERS.items():
    if col not in df_dem.columns:
        continue
    mask = pd.Series(False, index=df_dem.index)
    if lo is not None: mask |= df_dem[col] < lo
    if hi is not None: mask |= df_dem[col] > hi
    n = mask.sum()
    if n:
        flag_dem.append({'columna': col, 'unidad': unit,
                        'n_fuera_rango': n, 'rango_valido': f'[{lo}, {hi}]'})
        df_dem[col] = df_dem[col].clip(
            lower=lo if lo is not None else -np.inf,
            upper=hi if hi is not None else np.inf
        )

if flag_dem:
    print('\nValores fuera de rango detectados y recortados:')
    print(pd.DataFrame(flag_dem).to_string(index=False))
else:
    print('\nTodos los límites DEM superados...')

```

```

Cobertura DEM:
      cobertura_%  n_validos  n_nulos
dem_elevacion_m    99.1000      424      4
dem_pendiente_deg    99.1000      424      4
dem_orientacion_deg  99.1000      424      4

```

Todos los límites DEM superados...

## 5.- Fusión y Exportación

Todas las fuentes online se unen en un único DataFrame por `SITE_ID` y se exportan como `online_features_vx.csv` para unirlo con la salida del notebook 01.

```

In [59]: def _drop_empty_cols(df, etiqueta):
# Mantiene tu lógica original de limpieza de columnas vacías
cols_vacias = [c for c in df.columns if df[c].isna().all()]
if cols_vacias:
    print(f" [INFO] {etiqueta}: eliminadas {len(cols_vacias)} cols completamente vacías")
    return df.drop(columns=cols_vacias, errors='ignore')

plan_online = [
    (df_gee, 'gee'),
    (df_cds, 'cds'),
    (df_dem, 'dem'),
]

# 1. Aseguramos que la tabla base use siempre la clave de control 'site_id' en minúsculas
df_online = sites[['site_id']].copy()
df_online['site_id'] = df_online['site_id'].astype(str).str.strip()

print(f'Tabla base online inicial: {df_online.shape}')

```

```

for right, etiqueta in plan_online:
    right = _drop_empty_cols(right, etiqueta)

    # 2. Clonar localmente para evitar avisos de copia y detectar dinámicamente la columna id_sitio
    right = right.copy()
    col_sitio = [c for c in right.columns if c.lower() == 'site_id']

    if not col_sitio:
        print(f" [WARN] {etiqueta}: sin columna de identificación de sitio - omitido")
        continue

    col_sitio = col_sitio[0]

    # 3. Homogeneizar el nombre a 'site_id' y limpiar los tipos de datos a texto (str)
    if col_sitio != 'site_id':
        right = right.rename(columns={col_sitio: 'site_id'})

    right['site_id'] = right['site_id'].astype(str).str.strip().str.replace('.', '', regex=False)

    # 4. Fusionar con la clave normalizada 'site_id'
    antes = df_online.shape[1]
    df_online = df_online.merge(right, on='site_id', how='left',
                               suffixes=('', f'_{etiqueta}'))

    print(f' + {etiqueta:6s} → +{df_online.shape[1]-antes:3d} cols (total: {df_online.shape[1]})')

```

```

Tabla base online inicial: (428, 1)
+ gee   → + 3 cols (total: 4)
[INFO] cds: eliminadas 1 cols completamente vacías
+ cds   → + 0 cols (total: 4)
+ dem   → + 3 cols (total: 7)

```

```

In [60]: # Resumen de cobertura global
online_cols = [c for c in df_online.columns if c != 'site_id']
cobertura_total = pd.concat([cobertura_gee, cobertura_cds, cobertura_dem])
print('Cobertura total - todas las variables online:')
print(cobertura_total.sort_values('cobertura_%').to_string())

bajas = cobertura_total[cobertura_total['cobertura_%'] < 80]
if len(bajas):
    print(f'\n[WARN] {len(bajas)} variables con cobertura < 80%:')
    print(bajas.to_string())

```

```

Cobertura total - todas las variables online:
      cobertura_%  n_validos  n_nulos
cds_precip_mm_mes      0.0000         0      428
gee_ndvi_verano       97.9000       419         9
dem_pendiente_deg     99.1000       424         4
dem_elevacion_m       99.1000       424         4
dem_orientacion_deg   99.1000       424         4
gee_humedad_rel_pct   99.8000       427         1
gee_temp_media_C      99.8000       427         1

```

```

[WARN] 1 variables con cobertura < 80%:
      cobertura_%  n_validos  n_nulos
cds_precip_mm_mes      0.0000         0      428

```

```

In [61]: # Exportar
online_path = OUT_DIR + 'online_features_v3.csv'
df_online.to_csv(online_path, index=False)
print(f' {online_path} done...')
print(f' {df_online.shape[0]} sitios × {df_online.shape[1]} columnas')
print()
print('Columnas exportadas:')
for col in df_online.columns:
    fuente = 'GEE' if col.startswith('gee_') else 'CDS' if col.startswith('cds_') else 'DEM' if col.startswith('dem_') else 'ID'
    print(f' [{fuente:3s}] {col}')

```

```

output/online_features_v3.csv done...
428 sitios × 7 columnas

```

```

Columnas exportadas:
[ ID ] site_id
[ GEE ] gee_temp_media_C
[ GEE ] gee_humedad_rel_pct
[ GEE ] gee_ndvi_verano
[ DEM ] dem_elevacion_m
[ DEM ] dem_pendiente_deg
[ DEM ] dem_orientacion_deg

```